

Schwarzschild Metric from Scratch via $\mathcal{UD}$ Compilation

Brian Beckman

Feb 2026

Abstract

The question to answer in this notebook is “what is the vacuum spacetime geometry outside a static, spherically symmetric, non-spinning black hole?”

The Schwarzschild metric is the answer. In this notebook, we derive it *ab-initio*. The derivation is certainly not novel, but the novel contribution here is advancement of the $\mathcal{UD}$ calculus, moving its compiler from proof-of-concept to prototype. This metric is demonstrated in another notebook, *[Interactive black hole dynamics: Schwarzschild orbits via UD calculus](#)*.

The $\mathcal{UD}$ calculus automates symbolic operations such as covariant differentiation, tensor quotienting, canonicalizing indices, etc. The $\mathcal{UD}$ compiler translates between index notations in $\mathcal{UD}$ and component representations in Wolfram arrays.

- A script version of this notebook, minus the prose, is available at https://github.com/rebcabin/RelativityToolkit/blob/main/schwarzschild_from_scratch.wl.

Audience

This series of notebooks is targeted at readers with an undergraduate’s STEM education—engineering, computer science, mathematics. Exposure to introductory courses in Classical Mechanics (e.g., *[Goldstein](#)*), Electrodynamics (e.g., *[Jackson](#)*), and Relativity (e.g., *[Schutz](#)*) will help.

- With the exception of MTW, I do not put standard textbooks in the References section, but instead offer clickable (non-affiliate!) links in the prose of this notebook.

Initialize the Relativity Toolkit

The toolkit is currently implemented as a script rather than as a Package, prioritizing development flexibility and transparency over encapsulation. All symbols, including internal variables in Module constructs, are loaded into the Global namespace. That makes it difficult to find just the intentional exports of the toolkit without reading the *[code](#)* or the *[reference manual](#)*. This inconvenience will be

mitigated in future versions.

Optional: Quit for a Fresh Kernel

- *Note, if you call `Quit` in a notebook, evaluation may hang. If you want to “Evaluate Notebook,” mark the following two cells “Evaluatable” in the Cell→Property menu, call `Quit` and `FrontEndTokenExecute` once each, then comment out the cells, or mark them unevaluatable again. Alternatively, you can quit the kernel via the Evaluation menu (last item). At that point, you can “Evaluate Notebook” in a clean environment. Otherwise, you can evaluate cells in this notebook individually.*

```
In[*]:= Quit[];

In[*]:= FrontEndTokenExecute["DeleteGeneratedCells"]
```

Set Download Loci and Load Main Script

```
In[*]:= RTbranch = "v1.11.0";
RTbase = "https://raw.githubusercontent.com/rebcabin/RelativityToolkit/" <>
  RTbranch <> "/";
RTbust = "?t=" <> ToString[Round[AbsoluteTime[]]];
(*Force fresh download*)
RTrules = RTbase <> "RelativityToolkit.wl" <> RTbust;
RTtests = RTbase <> "RelativityToolkit_RegressionTests.wl" <> RTbust;
RTviz = RTbase <> "RelativityToolkit_VisualShowcase.wl" <> RTbust;
Get[RTrules];

» Relativity Toolkit version : 1.11.0
» Relativity Connection Symbol :  $\Gamma$ 
```

Optional: See all Tests and Results from Prior Works

Make any of the following cells evaluatable if you wish. They’re unevaluatable by default to keep this notebook trim.

```
Get[RTtests];

In[*]:= Get[RTviz] (* no semicolon, on purpose *)

In[*]:= Get[RTbase <> "covariant_derivative.wl"]

In[*]:= Get[RTbase <> "law_of_gamma.wl"]

Get[RTbase <> "connections_to_christoffels.wl"]

Get[RTbase <> "calculate_riemann.wl"]
```

Optional: Configure Relativity Connection Symbol

To properly compose covariant derivatives, $\mathcal{UD}$ must know the user’s reserved symbols for non-tensorial objects like connection coefficients. In gravitational physics, these force-producing connec-

tions are called Γ (capital gamma). Other theories like electrodynamics and Yang-Mills use other connection symbols.

We're going with Γ for this relativity/gravitation notebook, it's the default, so the following cell is optional.

```
In[ ]:= SetConnection[Γ];
Relativity Engine: Connection set to Γ
```

Solution Strategy

The story of Karl Schwarzschild's solution, while he was enlisted in the Prussian Army in WWI, of the Einstein field equations very soon after their publication in 1915 is fascinating. Perhaps Karl followed a strategy like the following:

1. Guess at the form of the Schwarzschild metric, $g_{\mu\nu}$: that is, make an *ansatz*.
2. Grind this form through new compiler components to generate (1) Christoffel symbols, (2) the Riemann curvature tensor $R^\mu_{\lambda\nu\rho}$, and finally (3) the Ricci tensor $R_{\mu\nu}$, which is a specific contraction of Riemann.

3. Consult the Einstein field equations (not derived in this notebook):

$$R_{\mu\nu} - \frac{1}{2} R g_{\mu\nu} = \frac{8\pi G}{c^4} T_{\mu\nu} \quad (1)$$

where R is the trace of Ricci, namely $g^{\mu\nu} R_{\mu\nu}$.

4. Set the Einstein equations equal to zero for the vacuum ($T_{\mu\nu} = 0$, no stress-energy).
5. Contract the Einstein equations against $g^{\mu\nu}$ to get the Ricci trace, R :

$$g^{\mu\nu} R_{\mu\nu} - \frac{1}{2} R (g^{\mu\nu} g_{\mu\nu}) = R - \frac{1}{2} \times 4 R = 0 \implies R = 0 \quad (2)$$

6. Substitute this back into the Einstein equations to deduce that the all components of Ricci vanish in the vacuum:

$$R_{\mu\nu} - \frac{1}{2} (0) g_{\mu\nu} = 0 \implies R_{\mu\nu} = 0 \quad (3)$$

7. Derive an ODE for terms in the ansatz from vanishing Ricci components.
8. To solve the ODE, invoke boundary conditions at infinity, and invoke a Newtonian far-field approximation to evaluate constants of integration.
9. Compare against Wolfram's stored resource.

■ I was fortunate to have Schwarzschild's son, Martin, as one of my advisors in graduate school.

Introducing New Compiler Functions

$\mathcal{UD}$ has been enhanced with several new functions in service of this task. The following paragraphs sketch their functionality, and the rest of the notebook illustrates their use by example. These func-

tions are not specific to this task, however. In the future, we'll need them for the Kerr metric (spinning black holes), as well as for other problems in general relativity, gauge theory, and differential geometry. In this notebook, "matrix" is a synonym for a Wolfram 2D array. A *tensor* is either a matrix that satisfies certain coordinate-transformation rules or a collection of symbolic $\mathcal{UD}$ expressions that satisfy those same coordinate-transformation rules.

MatrixToUDRules

Converts the non-zero elements of a matrix of components into rules for later application, via `Replace` and `ReplaceAll`, to symbolic expressions in $\mathcal{UD}$.

ChristoffelsFromMetric

Computes torsion-free Christoffel symbols from a symmetric metric tensor written in $\mathcal{UD}$ notation.

Contract

Contracts a given bound index in a $\mathcal{UD}$ expression over a given set of coordinate symbols. Analogous to Wolfram built-in `Replace`.

In[]:= **ContractAll**

Contracts all bound indices in a $\mathcal{UD}$ expression over a given set of coordinate symbols. Analogous to Wolfram built-in `ReplaceAll`.

In[]:= **EvaluateUDPartials**

Converts (compiles) partial derivatives $\mathcal{UD}$ notation into forms that Wolfram can manipulate.

In[]:= **MakeIndexer**

Makes a function that can index a matrix via symbolic coordinates rather than by `Part` numbers.

In[]:= **CalculateRiemannComponent**

Given Christoffel symbols via an indexer, calculate a component of the Riemann curvature tensor, specified via natural coordinate symbols, by contracting over the Christoffel symbols.

In[]:= **CalculateRicciComponent**

Given a Riemann tensor via an indexer, calculate a component of the Ricci tensor, a particular contraction of the Riemann tensor. Specify the desired component via natural coordinate symbols.

See the source file [RelativityToolkit.wl](#) for implementation details.

Ansatz

Set up a coordinate system in flat spacetime, at a safe distance from the central black hole, but with origin at the center of the black hole. Measure all events with respect to this coordinate system. Curvature of actual spacetime, as opposed to coordinate system, are accounted for by solving equations of motion with respect to proper time.

Assume as a guess, or *ansatz*, the following form for an element of proper distance along a geodesic:

$$ds^2 = -A(r) dt^2 + B(r) dr^2 + r^2 d\theta^2 + r^2 \sin^2(\theta) d\phi^2 \equiv g_{tt} dt^2 + g_{rr} dr^2 + g_{\theta\theta} d\theta^2 + g_{\phi\phi} d\phi^2 \quad (4)$$

with two unknown functions of radius, $A(r)$ and $B(r)$, and with symbolic coordinates, t , r , θ , ϕ , all assumed to be functions of proper time, τ , the time experienced by a particle in free-fall on a geodesic.

Particularly, $t(\tau)$, $r(\tau)$, ..., $A(r(\tau))$, $B(r(\tau))$ are all functions of time. Notice these functions do not depend on the angle coordinates θ and ϕ . That is the meaning of “spherically symmetric.” They also do not depend on coordinate time, and that is the meaning of “static.”

Covariant Metric Tensor

Express the covariant metric tensor $g_{\mu\nu}$ of this ansatz as a diagonal matrix (Wolfram array); save the coordinates for later:

```
In[ ]:= Echo[(SchwAnsatz = DiagonalMatrix[{-A[r], B[r], r^2, r^2 Sin[θ]^2}]) // MatrixForm,
  "Schw ansatz :"];
coords = {t, r, θ, φ};
» Schw ansatz : 
$$\begin{pmatrix} -A[r] & 0 & 0 & 0 \\ 0 & B[r] & 0 & 0 \\ 0 & 0 & r^2 & 0 \\ 0 & 0 & 0 & r^2 \sin[\theta]^2 \end{pmatrix}$$

```

- Coordinate time, t , is also a function of proper time, i.e., it's $t(\tau)$, expressing time dilation.
- The g_{tt} component of $g_{\mu\nu}$ is negative because we adopt the $(-1, 1, 1, 1)$ sign convention.

Metric Rules

Call the new compiler function, **MatrixToUDRules**, to create rules that map $\mathcal{UD}$ -indexed components of a given symbol, say **gDD**, to the components of the ansatz matrix. We'll need these rules later to concretize abstract expressions.

```
In[ ]:= Echo[SchwRules = MatrixToUDRules[SchwAnsatz, gDD, d, coords], "Schw rules :"];
» Schw rules : {gDD t t → -A[r], gDD r r → B[r], gDD θ θ → r^2, gDD φ φ → r^2 Sin[θ]^2}
```

- For brevity, **MatrixToUDRules** doesn't produce rules for zeros in the matrix. We'll put those in later, by hand, in a concise way that exploits structural knowledge of g .
- TODO: Sparse-matrix version of **MatrixToUDRules** that handles zero elements without manual intervention.

Contravariant Metric Tensor and Rules

The contravariant version, $g^{\mu\nu}$, of g is just the matrix inverse of the covariant version $g_{\mu\nu}$ (because $g^{\mu\rho} g_{\rho\nu} = \delta^\mu_\nu$). Ending with the needed rules for **gUU**, we get:

```
In[ ]:= SchwAnsatzInv = Inverse[SchwAnsatz];
Echo[SchwInvRules = MatrixToUDRules[SchwAnsatzInv, gUU, u, coords],
  "Schw Inv rules :"];
» Schw Inv rules : {gUU t t → -1/A[r], gUU r r → 1/B[r], gUU θ θ → 1/r^2, gUU φ φ → Csc[θ]^2/r^2}
```

Christoffel Symbols

The functions developed in an earlier notebook, *Riemann Curvature in $\mathcal{UD}$ + Compiler POC*, are captured in a new compiler function, **ChristoffelsFromMetric**, which computes the required commuta-

tor of covariant derivatives. See it exposed here as an unevaluated contraction over a bound `Unique` index:

```
In[ ]:= Echo[ChristoffelsFromMetric[gDD, gUU, σ, μ, ν], "Unevaluated Contraction :"];
```

» Unevaluated Contraction : $\frac{1}{2} (g_{UU}^{\sigma \lambda 11}) (g_{DD}^{\lambda 11}{}_{\mu, \nu} + g_{DD}^{\lambda 11}{}_{\nu, \mu} - g_{DD}^{\mu \nu, \lambda 11})$

The new compiler function **ContractAll** expands a contraction as a sum over all coordinates. The up-down index pairs are now coordinate symbols rather than summation indices. Each term in the following exhibit is a product of concrete components, not a further (recursive) contraction. We can tell not to contract when a coordinate symbol, rather than a (usually Greek) index appears in up-down, paired index positions.

```
In[ ]:= ContractAll[ChristoffelsFromMetric[gDD, gUU, σ, μ, ν], coords]
```

```
Out[ ]:=
```

$$\begin{aligned} & \frac{1}{2} (g_{UU}^{\sigma r}) (g_{DD}^r{}_{\mu, \nu}) + \frac{1}{2} (g_{UU}^{\sigma r}) (g_{DD}^r{}_{\nu, \mu}) + \frac{1}{2} (g_{UU}^{\sigma t}) (g_{DD}^t{}_{\mu, \nu}) + \\ & \frac{1}{2} (g_{UU}^{\sigma t}) (g_{DD}^t{}_{\nu, \mu}) + \frac{1}{2} (g_{UU}^{\sigma \theta}) (g_{DD}^{\theta}{}_{\mu, \nu}) + \frac{1}{2} (g_{UU}^{\sigma \theta}) (g_{DD}^{\theta}{}_{\nu, \mu}) - \\ & \frac{1}{2} (g_{UU}^{\sigma r}) (g_{DD}^{\mu \nu, r}) - \frac{1}{2} (g_{UU}^{\sigma t}) (g_{DD}^{\mu \nu, t}) - \frac{1}{2} (g_{UU}^{\sigma \theta}) (g_{DD}^{\mu \nu, \theta}) - \\ & \frac{1}{2} (g_{UU}^{\sigma \phi}) (g_{DD}^{\mu \nu, \phi}) + \frac{1}{2} (g_{UU}^{\sigma \phi}) (g_{DD}^{\phi}{}_{\mu, \nu}) + \frac{1}{2} (g_{UU}^{\sigma \phi}) (g_{DD}^{\phi}{}_{\nu, \mu}) \end{aligned}$$

Inspect One Component

Now we can see clearly the reason for the rules above:

```
In[ ]:= Echo[TrttX = ContractAll[ChristoffelsFromMetric[gDD, gUU, r, t, t], coords] /.
```

```
  SchwRules /.
```

```
  SchwInvRules, "Trtt ="];
```

» $r_{tt} = -\frac{(-A[r])_{,r}}{2B[r]} + \frac{1}{2} (g_{UU}^{r t}) (-A[r])_{,t} - \frac{1}{2} (g_{UU}^{r \theta}) (-A[r])_{,\theta} -$

$$\frac{1}{2} (g_{UU}^{r \phi}) (-A[r])_{,\phi} + \frac{g_{DD}^r{}_{t,t}}{B[r]} + (g_{UU}^{r \theta}) (g_{DD}^{\theta}{}_{t,t}) + (g_{UU}^{r \phi}) (g_{DD}^{\phi}{}_{t,t})$$

Notice the unevaluated partial derivatives in comma notation, $-A[r]_{,r}$ and $A[r]_{,t}$, $g_{DD}^{r t,t}$, etc.

Kill Off-Diagonal Elements

Here are the promised extra rules to zero-out the off-diagonal components:

```
In[ ]:= offDiagonalRules = {
```

```
  gUU[u[a_], u[b_]] /; a != b → 0,
```

```
  gDD[d[a_], d[b_]] /; a != b → 0};
```

■ Recall that *MatrixToUDRules* does not currently do this, for practical reasons.

```
In[*]:= Echo[(Γtt = ΓttX) /. offDiagonalRules, "Γtt diag only ="];
```

$$\gg \Gamma_{tt}^{\text{diag only}} = \frac{0, t}{B[r]} - \frac{(-A[r]), r}{2 B[r]}$$

Notice, again, the unevaluated derivatives. $\mathcal{UD}$ doesn't evaluate derivatives until you tell it to do so.

- *Killing the off-diagonal terms is not strictly necessary in this step because the next function, EvaluateUDPartials, would have zeroed them out anyway. But it's conceptually cleaner this way.*

Evaluate $\mathcal{UD}$ Partial Derivatives

Another new $\mathcal{UD}$ compiler function, **EvaluateUDPartials**, produces a factor of $A'[r] = \partial A / \partial r = A[r]_{,r}$, in terms of the tick notation that Wolfram understands. We'll wrestle this into differential equations for A .

```
In[*]:= Echo[ΓttA = EvaluateUDPartials[ΓttY], "Γtt evaluated ="];
```

$$\gg \Gamma_{tt}^{\text{evaluated}} = \frac{A'[r]}{2 B[r]}$$

All Γ Components

Now that we're happy with a reduction strategy, apply it to all components of Γ in a new 3D matrix, one dimension for each index:

```
In[*]:= (SchwΓ = Table[
  (ContractAll[ChristoffelsFromMetric[gDD, gUU, σ, μ, ν], coords] /.
    SchwRules /. SchwInvRules /. offDiagonalRules) // EvaluateUDPartials,
  {σ, coords}, {μ, coords}, {ν, coords}]] //
Echo[# // MatrixForm, "Ansatz Γ ="] &;
```

$$\gg \text{Ansatz } \Gamma = \begin{pmatrix} \begin{pmatrix} 0 \\ \frac{A'[r]}{2 A[r]} \\ 0 \\ 0 \end{pmatrix} & \begin{pmatrix} \frac{A'[r]}{2 A[r]} \\ 0 \\ 0 \\ 0 \end{pmatrix} & \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \end{pmatrix} & \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \end{pmatrix} \\ \begin{pmatrix} \frac{A'[r]}{2 B[r]} \\ 0 \\ 0 \\ 0 \end{pmatrix} & \begin{pmatrix} 0 \\ \frac{B'[r]}{2 B[r]} \\ 0 \\ 0 \end{pmatrix} & \begin{pmatrix} 0 \\ 0 \\ -\frac{r}{B[r]} \\ 0 \end{pmatrix} & \begin{pmatrix} 0 \\ 0 \\ 0 \\ -\frac{r \sin[\theta]^2}{B[r]} \end{pmatrix} \\ \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \end{pmatrix} & \begin{pmatrix} 0 \\ 0 \\ \frac{1}{r} \\ 0 \end{pmatrix} & \begin{pmatrix} 0 \\ \frac{1}{r} \\ 0 \\ 0 \end{pmatrix} & \begin{pmatrix} 0 \\ 0 \\ 0 \\ -\cos[\theta] \sin[\theta] \end{pmatrix} \\ \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \end{pmatrix} & \begin{pmatrix} 0 \\ 0 \\ 0 \\ \frac{1}{r} \end{pmatrix} & \begin{pmatrix} 0 \\ 0 \\ 0 \\ \cot[\theta] \end{pmatrix} & \begin{pmatrix} 0 \\ \frac{1}{r} \\ \cot[\theta] \\ 0 \end{pmatrix} \end{pmatrix}$$

- *Here we really needed to kill the off-diagonal terms manually to produce a visually tolerable result.*

Γ Indexer

To reduce opportunities for human error, we really want to index objects with coordinate symbols rather than with numerical `Part` or `levelspec` numbers. We'd rather write `rget[r,t,t]` than `r[[1,0,0]]`.

MakeIndexer is the new compiler function that makes a function that maps symbolic coordinates to `Part` expressions. Test `rget` on an arbitrary component:

```
In[*]:= Echo[(rGet = MakeIndexer[SchwΓ, coords])[r, r, r], "Γrr ="];
» Γrr =  $\frac{B'[r]}{2 B[r]}$ 
```

The Riemann Ansatz in Full

Call another new compiler function, **CalculateRiemannComponent**, passing in `rget` and the coordinate symbols. Iterate symbolic indices μ, λ, ν , and ρ over all 256 combinations of the coordinates. There is far too much to inspect, by hand, here, but it's instructive to see some of it.

```
In[*]:= Echo[(SchwRiemann = Table[
  CalculateRiemannComponent[μ, λ, ν, ρ, rGet, coords],
  {μ, coords}, {λ, coords}, {ν, coords}, {ρ, coords}]] // Short[#, 5] &,
  "Rλνρμ ="];
» Rλνρμ = { { { {0, 0, 0, 0}, {0, 0, 0, 0}, {0, 0, 0, 0}, {0, 0, 0, 0}},
  { {0, - $\frac{A[r] A'[r] B'[r] + B[r] (A'[r]^2 - 2 A[r] A''[r])}{4 A[r]^2 B[r]}$ , 0, 0},
  {  $\frac{A[r] A'[r] B'[r] + B[r] (\ll 1 \gg^2 - 2 \ll 1 \gg \ll 1 \gg)}{4 A[r]^2 B[r]}$ , 0, 0, 0}, {0, 0, 0, 0}, {0, 0, 0, 0}},
  { {0, 0,  $\frac{r \ll 1 \gg}{2 \ll 1 \gg \ll 1 \gg \ll 1 \gg}$ , 0}, <<3>>}, { {0, 0, 0,  $\frac{r \sin[\theta]^2 A'[r]}{2 A[r] \times B[r]}$ },
  {0, 0, 0, 0}, {0, 0, 0, 0}, { - $\frac{r \sin[\theta]^2 A'[r]}{2 A[r] \times B[r]}$ , 0, 0, 0} }}, <<2>>, { <<1>> } }
```

(Optional) Visualization

- Please tolerate some imprecision in this section for the sake of brevity.

View R_{abcd} as an operator that connects a spacetime $\{c, d\}$ Loop Plane to an $\{a, b\}$ Rotation Plane.

Parallel-transport a test vector around a loop in the $\{c, d\}$ plane. The test vector does not return to its original direction. In the $\{a, b\}$ rotation plane, measure the angular deficit against the original test vector.

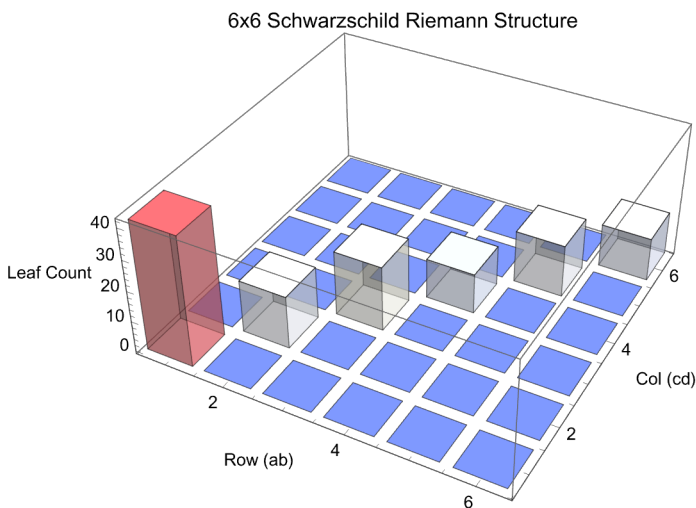
There are 6 orthogonal coordinate planes in spacetime $\{tx, ty, tz, xy, xz, yz\}$. The $\{tx, ty, tz\}$ planes express Lorentz boosts and the pure-space $\{xy, xz, yz\}$ planes express ordinary 3D rotations.

Map numerical index pairs to a single index $A = 1..6$ to get a *Petrov Matrix* (see reference). Measure the expression complexity for maps from $\{c, d\}$ loop-plane columns to $\{a, b\}$ rotation-plane rows.

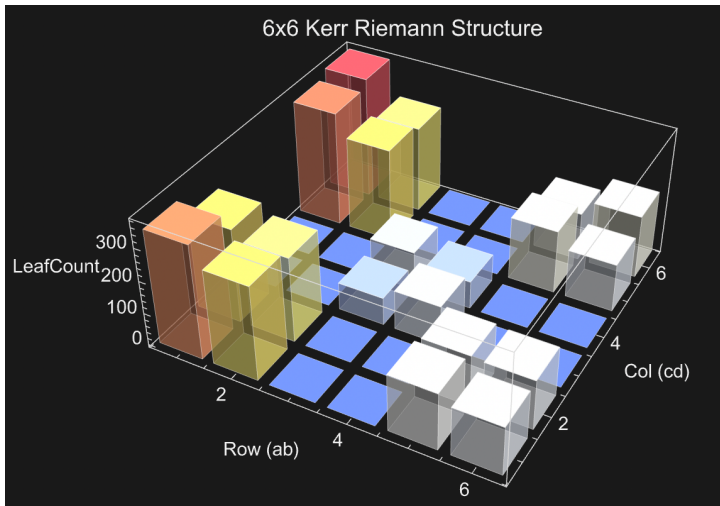
In the image, diagonal towers (A_{11} , A_{44} , etc.) represent “Gaussian curvature.” Walking a test vector in a circle in the $\{c, d\}$ plane rotates the vector within that same $\{c, d\}$ plane. This is the simple gravity we understand intuitively. Here is this Petrov visualization of our diagonal Schwarzschild ansatz:

```
In[ ] := petrovRiemannViz[R_] :=
Module[{
  bivectorMap = <|{1, 2} → 1, {1, 3} → 2, {1, 4} → 3, {2, 3} → 4, {2, 4} → 5, {3, 4} → 6|>,
  petrovMatrix = ConstantArray[0, {6, 6}],
  expr, val, row, col,
Do[
  Do[
    (expr = R[[i, j, k, l]];
    val = If[PossibleZeroQ[expr], 0, LeafCount[expr]];
    row = bivectorMap[Sort[{i, j}]];
    col = bivectorMap[Sort[{k, l}]];
    petrovMatrix[[row, col]] += Abs[val]);,
    {k, 1, 3}, {l, k + 1, 4}],
    {i, 1, 3}, {j, i + 1, 4}];
DiscretePlot3D[petrovMatrix[[row, col]], {row, 1, 6}, {col, 1, 6},
  ExtentSize → 0.8,
  PlotRange → All,
  AxesLabel → {"Row (ab)", "Col (cd)", "Leaf Count"},
  ColorFunction → "TemperatureMap",
  PlotLabel → "6x6 Schwarzschild Riemann Structure"]];
petrovRiemannViz[SchwRiemann]
```

Out[] =



Off-diagonal towers (A_{14} , A_{26} , etc.) represent “gravitomagnetism” or the frame dragging of Kerr space-time, covered in [another notebook](#). It says, for example: “If I walk my vector around in the $\{x, y\}$ plane, it gets twisted into the $\{t, z\}$ plane.” Notice the much larger leaf counts overall. Kerr is much more complicated than Schwarzschild.



Top-left towers represent tidal forces (boosts caused by boosts). Bottom-right towers represent spatial curvature. Off-diagonal towers in the Kerr metric indicate that the black hole's rotation entwines these two sectors.

Riemann Indexer

As with Γ , index Riemann via coordinate symbols. Look at a couple of components and check a famous antisymmetry condition:

```
In[ ]:= Echo[(RGet = MakeIndexer[SchwRiemann, coords])[t, r, t, r], "Rtrtr ="];
```

$$\gg R_{rtr}^t = -\frac{A[r] A'[r] B'[r] + B[r] (A'[r]^2 - 2 A[r] A''[r])}{4 A[r]^2 B[r]}$$

```
In[ ]:= Echo[RGet[t, r, r, t], "Rrttr ="];
```

$$\gg R_{ttr}^r = \frac{A[r] A'[r] B'[r] + B[r] (A'[r]^2 - 2 A[r] A''[r])}{4 A[r]^2 B[r]}$$

```
In[ ]:= Echo[RGet[t, r, r, t] == -RGet[t, r, t, r]]
```

$\gg$ True

```
Out[ ]:=
```

True

The Ricci Ansatz

Build Ricci via another new compiler function, `CalculateRicciComponent`, which contracts Riemann over its single up index and its second down index. Pass in `Rget` and the coords, and iterate over the remaining 16 index values. This is big but inspectable.

```
In[ ]:= Echo[ (SchwRicci = Table[
  CalculateRicciComponent[λ, ρ, RGet, coords],
  {λ, coords}, {ρ, coords}]] // TraditionalForm, "Rλρ ="];
» Rλρ =
```

$$\begin{pmatrix} \frac{r B(r) A'(r)^2 + A(r) (r A'(r) B'(r) - 2 B(r) (r A''(r) + 2 A'(r)))}{4 r A(r) B(r)^2} & 0 \\ 0 & -\frac{A(r) (r A'(r) + 4 A(r)) B'(r) + r B(r) (A'(r)^2 - 2 A(r) A''(r))}{4 r A(r)^2 B(r)} \\ 0 & 0 \\ 0 & 0 \end{pmatrix} \quad \frac{\frac{r A'(r)}{2 A(r)} + 1}{B(r)}$$

The Einstein Vacuum Equations

There is a lot of structure in Ricci. In particular, the tt and rr components are very similar. Wrestle a tight, easy equation out of them.

Of course, we'll need a getter for Ricci.

```
In[ ]:= RicciGet = MakeIndexer[SchwRicci, coords];
Echo[Rtt = RicciGet[t, t], "Rtt ="];
Echo[Rrr = RicciGet[r, r], "Rrr ="];
» Rtt =
```

$$\frac{r B[r] A'[r]^2 + A[r] (r A'[r] B'[r] - 2 B[r] (2 A'[r] + r A''[r]))}{4 r A[r] B[r]^2}$$

```
» Rrr = -
```

$$\frac{A[r] (4 A[r] + r A'[r]) B'[r] + r B[r] (A'[r]^2 - 2 A[r] A''[r])}{4 r A[r]^2 B[r]}$$

Expand R_{tt} and R_{rr} to spot an opportunity to get rid of $A''[r]$:

```
In[ ]:= Rtt // Expand
Out[ ]:=
```

$$-\frac{A'[r]}{r B[r]} + \frac{A'[r]^2}{4 A[r] \times B[r]} + \frac{A'[r] B'[r]}{4 B[r]^2} - \frac{A''[r]}{2 B[r]}$$

```
In[ ]:= Rrr // Expand
Out[ ]:=
```

$$-\frac{A'[r]^2}{4 A[r]^2} - \frac{B'[r]}{r B[r]} - \frac{A'[r] B'[r]}{4 A[r] \times B[r]} + \frac{A''[r]}{2 A[r]}$$

By inspection, $\frac{R_{tt}}{A[r]} + \frac{R_{rr}}{B[r]}$ will do it:

```
In[ ]:= (Rtt/A[r] + Rrr/B[r]) // Expand
Out[ ]:=
```

$$-\frac{A'[r]}{r A[r] \times B[r]} - \frac{B'[r]}{r B[r]^2}$$

Relation between $A[r]$ and $B[r]$

Because all these terms vanish (every component of Ricci vanishes in the vacuum), we can solve for $B[r]$ in terms of $A[r]$:

```
In[*]:= Echo[DSolve[ $\frac{R_{tt}}{A[r]} + \frac{R_{rr}}{B[r]} == 0$ , B[r], r], "Relation between A and B :"];
```

» Relation between A and B : $\left\{ \left\{ B[r] \rightarrow \frac{c_1}{A[r]} \right\} \right\}$

Apply flat-space boundary conditions at infinity, meaning $\lim_{r \rightarrow \infty} A[r] = 1$ and $\lim_{r \rightarrow \infty} B[r] = 1$ to deduce that this $c_1 = 1$. We'll see another c_1 in the next step.

Final Equation and Solution

Drag out the $\theta\theta$ component and solve for a final differential equation for $A[r]$:

```
In[*]:= Echo[R $\theta\theta$  = RicciGet[ $\theta$ ,  $\theta$ ], "R $\theta\theta$  ="];
```

» $R_{\theta\theta} = -1 + \frac{1 + \frac{r A'[r]}{2 A[r]}}{B[r]} - \frac{r B'[r]}{2 B[r]^2}$

```
In[*]:= Echo[(finalODE = (R $\theta\theta$  == 0) /. {B[r] -> 1/A[r], B'[r] -> D[1/A[r], r]} // FullSimplify), "Vacuum ODE for A[r] :"];
```

» Vacuum ODE for A[r] : $A[r] + r A'[r] == 1$

```
In[*]:= Echo[SchwA = DSolve[finalODE, A[r], r], "Schwarzschild Solution :"];
```

» Schwarzschild Solution : $\left\{ \left\{ A[r] \rightarrow 1 + \frac{c_1}{r} \right\} \right\}$

We just need to figure out the new integration constant, c_1 .

Constant of Integration

Demand the **Newtonian limit**: that the black hole behave like a normal Newtonian star when we are far away from it.

In this weak-field, far-field limit, it is a standard result (not derived in this notebook) that the tt component of the metric, g_{tt} , be directly related to the Newtonian gravitational potential Φ :

$$g_{tt} \approx -(1 + 2\Phi) \quad (5)$$

in geometric units where $G = c = 1$.

For a star of mass M , $\Phi = -M/r$, therefore

$$g_{tt} \approx -\left(1 - \frac{2M}{r}\right) \quad (6)$$

Substituting our solution for $A[r]$ into the ansatz, we have

$$g_{tt} = -A[r] = -\left(1 + \frac{c_1}{r}\right) \quad (7)$$

Therefore $c_1 = -2M$.

Validation Against Wolfram Function Repository

Pulling together all results:

```
In[*]:= Echo[SchwAnsatz /. {B[r] → 1/A[r]} /. SchwA /. {c1 → -2 M},
  "Schwarzschild Solution via  $\mathcal{UD}$  :"];
```

» Schwarzschild Solution via $\mathcal{UD}$:

$$\left\{ \left\{ \left\{ -1 + \frac{2M}{r}, 0, 0, 0 \right\}, \left\{ 0, \frac{1}{1 - \frac{2M}{r}}, 0, 0 \right\}, \left\{ 0, 0, r^2, 0 \right\}, \left\{ 0, 0, 0, r^2 \sin[\theta]^2 \right\} \right\} \right\}$$

and comparing against the WFR resource

```
In[*]:= Echo[ResourceFunction["MetricTensor"]["Schwarzschild"]["MatrixRepresentation"],
  "Schwarzschild Solution via WFR :"];
```

» Schwarzschild Solution via WFR :

$$\left\{ \left\{ -1 + \frac{2M}{r}, 0, 0, 0 \right\}, \left\{ 0, \frac{1}{1 - \frac{2M}{r}}, 0, 0 \right\}, \left\{ 0, 0, r^2, 0 \right\}, \left\{ 0, 0, 0, r^2 \sin[\theta]^2 \right\} \right\}$$

we see a perfect match, except for WFR's convention of formal symbols (with underdots).

Conclusion

In this notebook, we derived the Schwarzschild metric for a static, spherically symmetric vacuum spacetime via $\mathcal{UD}$, the symbolic tensor calculator and compiler.

The primary contribution of this work is advancing $\mathcal{UD}$ for deeper calculations in general relativity. Compiler components in the toolkit translate a symbolic layer in index notation to a matrix layer for processing by existing Wolfram resources. Additional compiler functions support indexing by coordinate symbols rather than by row number and column number, for improved readability.

Validation points include:

- Symbolic Integrity: $\mathcal{UD}$ handles index valence, canonicalization, and contraction.
- Algebraic Simplification: $\mathcal{UD}$ generates the Riemann and Ricci tensors and decouples the vacuum field equations.
- Ground Truth Verification: The final derived solution matches the standard result in the Wolfram Function Repository exactly.

References

1. Beckman, Brian. *From the steam age to quantum fields: a unified approach via UD tensorial calculus*. Wolfram Community post.
2. Beckman, Brian. *General relativity in the UD calculus*. Wolfram Community post.

3. Beckman, Brian. *Formal Differential Geometry in the UD Calculus: Part 1*. Wolfram Community post.
4. Beckman, Brian. *Covariant Derivatives and Connections in the UD Calculus*. Wolfram Community post.
5. Beckman, Brian. *Riemann Curvature in UD + Compiler POC*. Wolfram Community post.
6. Beckman, Brian. *Schwarzschild Orbits in the UD Calculus*. Wolfram Community post.
7. Beckman, Brian. *A Wizard's Quest: Spinning Black Holes in UD*. Wolfram Community post.
8. Wolfram Function Repository. *MetricTensor*. For ground truth.
9. Wolfram Function Repository. *ChristoffelSymbol*. For ground truth.
10. [MTW] Misner, Charles W.; Thorne, Kip S.; Wheeler, John Archibald. *Gravitation*. Princeton University Press, 2017.
11. Petrov, A. Z. *Classification of spaces defining gravitational fields*, Scientific Transactions of Kazan State University, 114, 55 (1954)